DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (IOT)

Student Management System

A Java-Based Console Application

Using Core Java, OOP Principles & File Handling

SUBMITTED BY	
Name	Deblina Mondal
Roll Number	23
Enrollment Number	12024052019020
Section	A
Stream	CSE (IoT) — Internet of Things
Subject	Java Programming Project

1. Abstract

The Student Management System (SMS) is a console-based Java application developed using core Java programming concepts. The system provides a comprehensive solution for managing student academic records in an educational institution. It enables administrators and users to perform essential operations such as adding new student records, updating existing information, deleting records, searching for specific students, and displaying all stored data.

This project has been built entirely using Raw Java (Java SE), without any external frameworks or database libraries, relying on Java Collections Framework for in-memory data storage and Java File I/O for persistent data management. The application demonstrates the practical application of Object-Oriented Programming (OOP) principles including encapsulation, inheritance, polymorphism, and abstraction.

The primary goal of this project is to provide a functional, menu-driven interface that allows seamless management of student records while reinforcing foundational Java programming skills.

2. Introduction

In modern educational institutions, the management of student data is a critical administrative task. Manual record-keeping systems are error-prone, time-consuming, and difficult to scale. A computerized Student Management System offers a reliable, efficient, and scalable solution to these challenges.

This project implements a Student Management System using the Java programming language, specifically focusing on core Java (also known as Raw Java or Java SE — Standard Edition). The choice of Raw Java ensures that the project emphasizes fundamental programming concepts without relying on external frameworks, making it an ideal academic exercise.

2.1 Background

Java is one of the most widely used programming languages in the world, particularly in enterprise-level application development. Its platform independence (Write Once, Run Anywhere), strong OOP support, and rich standard library make it an excellent choice for building management systems. This project leverages:

- Java SE 17+ for core language features
- ArrayList and HashMap from the Java Collections Framework
- FileReader, FileWriter, and BufferedWriter for file-based persistence
- Scanner class for console-based user input
- Exception handling for robust error management

2.2 Problem Statement

Educational institutions often face challenges in maintaining and accessing student records efficiently. The specific problems addressed by this system include:

- Lack of a centralized system for student data management
- Difficulty in searching and filtering records quickly
- Risk of data loss due to manual record-keeping
- Inability to update or delete records efficiently
- Absence of a user-friendly interface for non-technical administrators

2.3 Scope of the Project

The Student Management System covers the following scope:

- Management of student personal and academic information
- CRUD operations: Create, Read, Update, Delete
- Search functionality by student ID, name, or course
- Persistent data storage using text file I/O
- Console-based, menu-driven user interface

3. Objectives

The primary objectives of this project are as follows:

1. To design and implement a student record management system using Raw Java.
2. To apply Object-Oriented Programming (OOP) principles effectively in a real-world scenario.
3. To utilize the Java Collections Framework (ArrayList, HashMap) for dynamic data management.
4. To implement file handling using Java I/O streams for persistent data storage and retrieval.
5. To develop a clean, modular code structure with well-defined classes and methods.
6. To create a user-friendly, console-based menu interface for ease of use.
7. To handle runtime exceptions and edge cases gracefully using try-catch blocks.
8. To lay a foundation for future enhancements such as GUI integration and database connectivity.

4. System Design & Architecture

4.1 System Architecture

The Student Management System follows a 3-Tier Architecture pattern, ensuring a clean separation of concerns:

Layer	Class / File	Responsibility
Presentation Layer	MainMenu.java	Displays menu, reads user input, navigates options
Business Logic Layer	StudentService.java, Validator.java	Processes operations, validates data, enforces rules
Data Access Layer	StudentDAO.java, FileManager.java	Manages data storage, file read/write operations
Model Layer	Student.java	Defines Student entity with fields and getters/setters

4.2 Class Design

The system is composed of the following key Java classes:

4.2.1 Student.java (Model Class)

The Student class serves as the entity/model class. It contains the following fields:

- int id — Unique identifier for each student
- String name — Full name of the student
- int age — Age of the student
- String course — Course/programme enrolled in
- double marks — Aggregate marks or CGPA
- String email — Email address of the student

The class implements encapsulation by declaring all fields as private and providing public getter and setter methods. It also overrides the toString() method for formatted console output.

4.2.2 StudentManager.java (Business Logic)

This class acts as the service layer. It holds an ArrayList<Student> as the primary data store and provides the following core methods:

- addStudent(Student s) — Adds a new student to the list and persists to file
- deleteStudent(int id) — Removes a student by ID after confirmation
- updateStudent(int id) — Updates student fields by ID
- searchById(int id) — Returns a Student object matching the given ID
- searchByName(String name) — Returns a list of students matching the name
- displayAll() — Prints all student records in formatted tabular output

4.2.3 FileManager.java (Data Access Layer)

Handles all file I/O operations for persistent storage. Key methods include:

- saveToFile(List<Student> list) — Writes all records to students.txt
- loadFromFile() — Reads stored records at application startup
- exportReport() — Generates a formatted text report of all records

4.2.4 MainMenu.java (Presentation Layer)

The entry point of the application. It displays the menu, accepts user input using Scanner, and delegates to StudentManager based on the chosen option. Contains the main() method.

4.2.5 Validator.java (Utility Class)

Provides static validation methods to ensure data integrity:

- validateName(String name) — Checks for non-empty and valid characters
- validateMarks(double marks) — Ensures marks are within 0–100 range
- isValidEmail(String email) — Basic email format validation

5. Features & Functionality

The Student Management System provides the following key features:

#	Feature	Description

1	Add Student	Register a new student with all relevant academic and personal details including ID, name, age, course, marks and email.
2	Update Student	Modify any field of an existing student record by entering their unique student ID.
3	Delete Student	Permanently remove a student record from the system after a confirmation prompt.
4	Search by ID	Instantly retrieve the complete record of a specific student using their unique student ID.
5	Search by Name	Find students by name using linear search; supports partial name matching.
6	Display All	Show all stored student records in a formatted tabular output in the console.
7	File Persistence	All data is automatically saved to and loaded from a local text file (students.txt).
8	Input Validation	User inputs are validated for type, range, and format before processing.
9	Error Handling	All operations are wrapped in try-catch blocks to handle exceptions gracefully.

6. Implementation

6.1 Technology Stack

- Language: Java SE 17 (Raw Java / Core Java)
- IDE: IntelliJ IDEA / Eclipse
- Data Storage: Java Collections Framework (ArrayList<Student>)
- Persistence: Java File I/O (FileReader, FileWriter, BufferedWriter, BufferedReader)
- User Input: java.util.Scanner
- Version Control: Git & GitHub
- JDK Version: Java Development Kit 17+

6.2 Sample Code Snippets

The following are representative code snippets from the implementation:

6.2.1 Student Model Class

```
public class Student {     private int id;     private String name;     private int age;     private String course;     private double marks;     private String email;
public Student(int id, String name, int age,                    String course, double marks, String email) {         this.id     = id;         this.name   = name;
this.age    = age;         this.course = course;         this.marks  = marks;
this.email  = email;     }      public int    getId()     { return id; }     public String getName()   { return name; }     public double getMarks()  { return marks; }
// ... other getters & setters      @Override     public String toString() {
return String.format("| %-5d | %-20s | %-3d | %-15s | %-6.2f |",                id, name, age, course, marks);     } }
```

6.2.2 Add Student Method

```
public void addStudent(Student student) {     try {         if
(searchById(student.getId()) != null) {             System.out.println("Error:
Student ID already exists.");             return;         }
students.add(student);         fileManager.saveToFile(students);
System.out.println("Student added: " + student.getName());     } catch (Exception e)
{         System.out.println("Error: " + e.getMessage());     } }
```

6.2.3 File Save Method

```
public void saveToFile(List<Student> students) {     try (BufferedWriter writer =
new BufferedWriter(new FileWriter(FILE_PATH))) {         for (Student s : students) {
writer.write(s.getId() + "," + s.getName() + ","                 + s.getAge() + "," +
s.getCourse() + ","                 + s.getMarks() + "," + s.getEmail());
writer.newLine();         }     } catch (IOException e) {
System.out.println("File error: " + e.getMessage());     } }
```

7. OOP Concepts Applied

This project actively demonstrates the four fundamental pillars of Object-Oriented Programming:

7.1 Encapsulation

All fields of the Student class are declared as private, and access is provided only through public getter and setter methods. This ensures data integrity and prevents unauthorized modification of fields directly.

7.2 Abstraction

Complex internal operations such as file reading, data parsing, and record validation are hidden behind simple method interfaces. The MainMenu class interacts with StudentManager without needing to know the internal implementation details.

7.3 Inheritance

While the base implementation uses a concrete Student class, the design supports extension through inheritance. For example, a PGStudent class could extend Student to add postgraduate-specific fields such as thesis title or research area, without modifying the parent class.

7.4 Polymorphism

The toString() method is overridden in the Student class to provide a formatted, tabular string representation. This allows polymorphic behavior where objects can be processed uniformly through the common Object interface.

8. Testing

8.1 Test Cases

#	Test Case	Input	Expected Output	Status
1	Add Student	Valid student data	Record added and saved to file	PASS

2	Duplicate ID	Existing student ID	Error: ID already exists	PASS
3	Search by ID	Valid ID (e.g., 101)	Correct student record displayed	PASS
4	Invalid Search ID	Non-existent ID (999)	Student not found message	PASS
5	Delete Student	Valid ID + confirmation	Record deleted, file updated	PASS
6	Invalid Marks	Marks = -5 or 110	Validation error displayed	PASS
7	Empty Name	Name = ""	Validation error displayed	PASS
8	File Load	Application restart	All records loaded from file	PASS
9	Display All	No input needed	Formatted table of all students	PASS
10	Update Record	Valid ID, new values	Record updated in memory & file	PASS

9. Future Enhancements

While the current implementation is functional and demonstrates core Java concepts effectively, the following enhancements are planned for future development:

Phase 1 — Short Term

- Graphical User Interface (GUI) using Java Swing or JavaFX
- MySQL database integration using JDBC for robust data management
- User authentication system with login and role-based access

Phase 2 — Medium Term

- Attendance tracking and reporting module
- CGPA and grade calculator with grade distribution report
- PDF report generation using the iText library
- Export data to CSV/Excel format for administrative use

Phase 3 — Long Term

- Web-based version developed using Spring Boot framework
- RESTful API to support mobile application integration
- Cloud deployment on AWS, Azure, or Google Cloud Platform
- Integration with IoT devices for smart classroom management (aligned with CSE IoT stream)

10. Conclusion

The Student Management System project successfully demonstrates the power and versatility of Raw Java in building practical, real-world applications. Through this project, the core concepts of Java programming — including Object-Oriented Programming, the Collections Framework, File I/O, exception handling, and console-based interface design — have been applied in a cohesive and meaningful way.

The system provides a fully functional solution for managing student academic records, with features for adding, updating, deleting, searching, and displaying data. The use of file-based persistence ensures that data is retained across multiple sessions, making the application practical for real use.

This project has significantly strengthened understanding of software design principles such as modularity, encapsulation, and separation of concerns. It also provides a solid foundation for future enhancements, including GUI integration, database connectivity, and web-based deployment using modern Java frameworks.

11. References

9. Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.
10. Eckel, B. (2006). Thinking in Java (4th ed.). Prentice Hall.
11. Oracle Corporation. Java SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/
12. Horstmann, C. S. (2019). Core Java Volume I — Fundamentals (11th ed.). Pearson Education.
13. Deitel, P. & Deitel, H. (2017). Java How to Program (11th ed.). Pearson Education.
14. GeeksforGeeks. Java Collections Framework. https://www.geeksforgeeks.org/collections-in-java-2/
15. TutorialsPoint. Java File Handling. https://www.tutorialspoint.com/java/java_files_io.htm

Declaration

I hereby declare that this project report entitled "Student Management System" is my own original work, submitted in partial fulfillment of the requirements of the Java Programming course. This report has not been submitted elsewhere for any other degree or examination.

Name: Deblina Mondal

Roll No: 23 **Enrollment:** 12024052019020

Section: A **Stream:** CSE (IoT)

Signature: _____

Date: _____